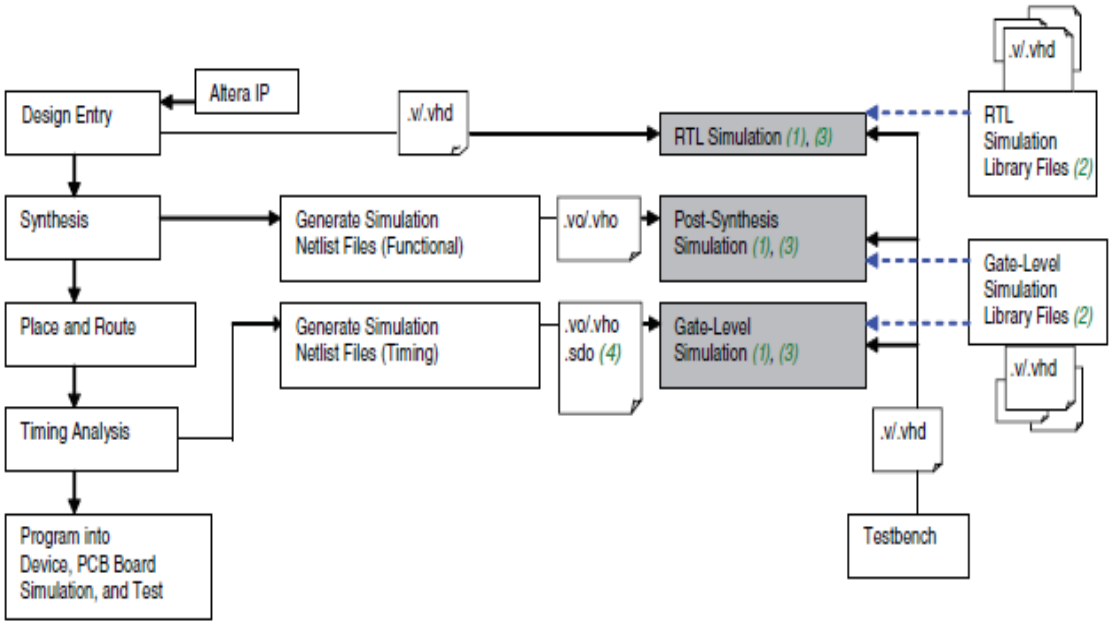
Introduction

Prior to manufacturing, the designer of a digital system must verify that the system satisfies all the requirements in the specification. To ensure that the design satisfies the requirements, the designer must do extensive simulation on the all system components, submodules, and the eventually the entire system itself. Prior versions of Quartus (version 9 or lower) offer built-in graphical simulation; starting from version 10, the simulation must be done using third party EDA (Electronic Design Automation) simulation tools. We shall illustrate the Altera design flow using EDA simulator and Quartus and describe the use of test benches in simulation. We shall describe the procedure to perform simulation for the Altera design using Modelsim simulator and the Quartus® II NativeLink feature in Tutorial 1, Tutorial 2, and Tutorial 3.

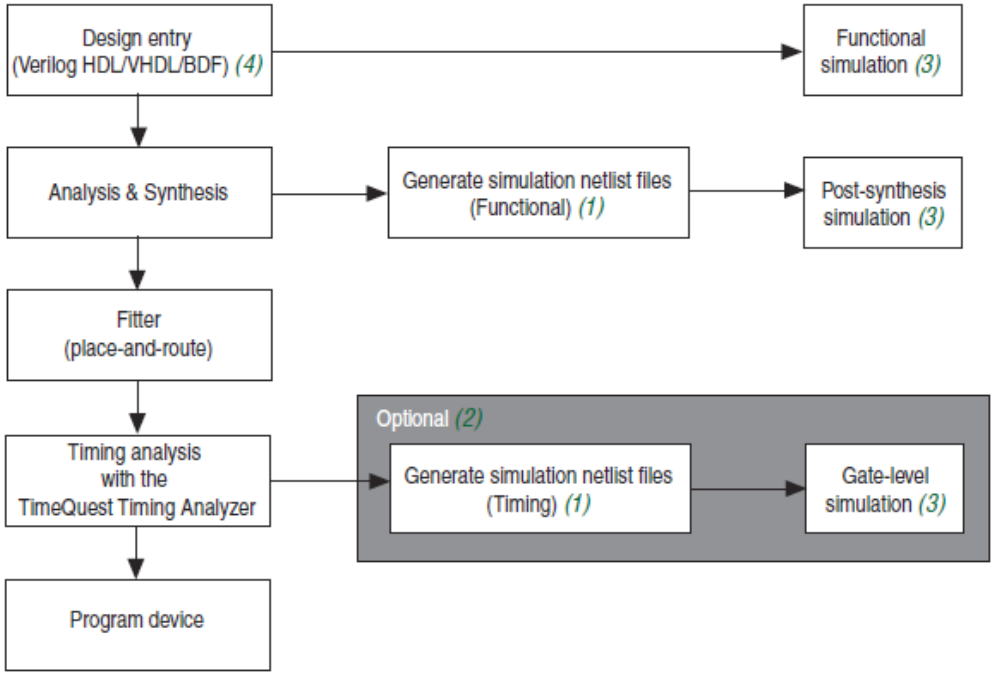
Figure 1–1. Altera Design Flow Using EDA Simulators and the Quartus II Software **Version 10**



Notes to Figure 1–1:

(1) Simulation can be run with the NativeLink feature or performed manually.
(2) ModelSim-Altera does not require simulation library source files for simulation.
(3) The simulation tools include ModelSim, ModelSim-Altera, QuestaSim, VCS, VCS MX, NCSim, Active-HDL, and Riviera-PRO.
(4) **.vo**, **.vho**, and **.sdo** are generated by the Quartus II Netlist Writer.

Figure 1–1. Altera Design Flow Incorporating Simulation **Version 11**



Notes to Figure 1–1:

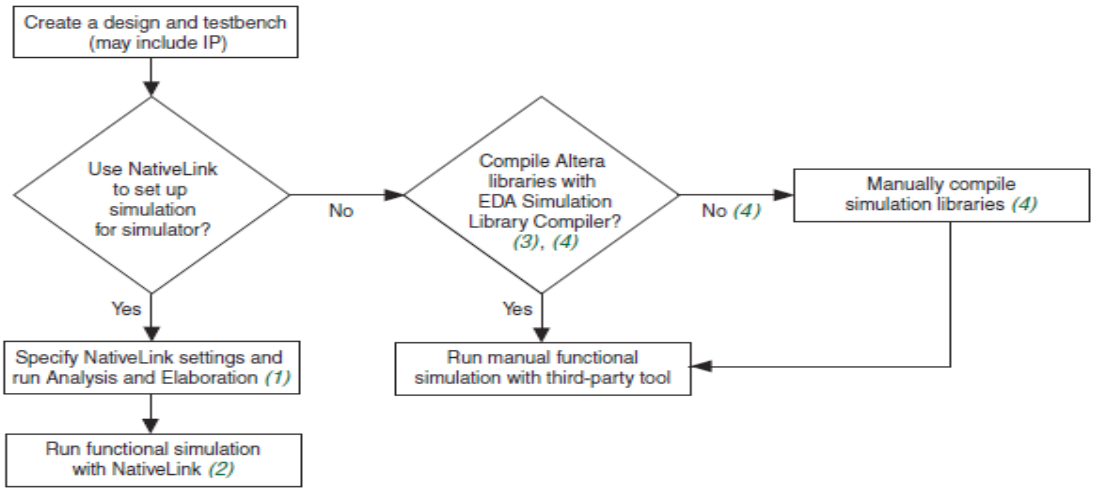
(1) Generate Verilog Output Files (**.vo**), VHDL Output Files (**.vho**), Standard Delay Format Output Files (**.sdo**), and SystemVerilog Files (**.svo**) with the Quartus II Netlist Writer.

(2) Altera recommends that you use the Quartus II TimeQuest Timing Analyzer to help verify and close timing. You also have the option to run post-fit timing simulation, which can be slow for a large design. For more information about the TimeQuest analyzer, refer to *The TimeQuest Timing Analyzer* chapter in volume 3 of the *Quartus II Handbook*.

(3) You can use the NativeLink feature to automate the process of running EDA simulations from the Quartus II software.

(4) You cannot use the Block Design File (**.bdf**) to perform functional simulation, because the format is not compatible with third-party HDL simulators (for example, ModelSim). The **.bdf** must be converted to HDL format (Verilog HDL or VHDL) before it can be used for functional simulation. For more information, refer to "Converting Block Design Files (.bdf) to HDL Format (.v/.vhd)" on page 1–4.

Figure 1–2. Functional Simulation Flow



Notes to Figure 1–2:

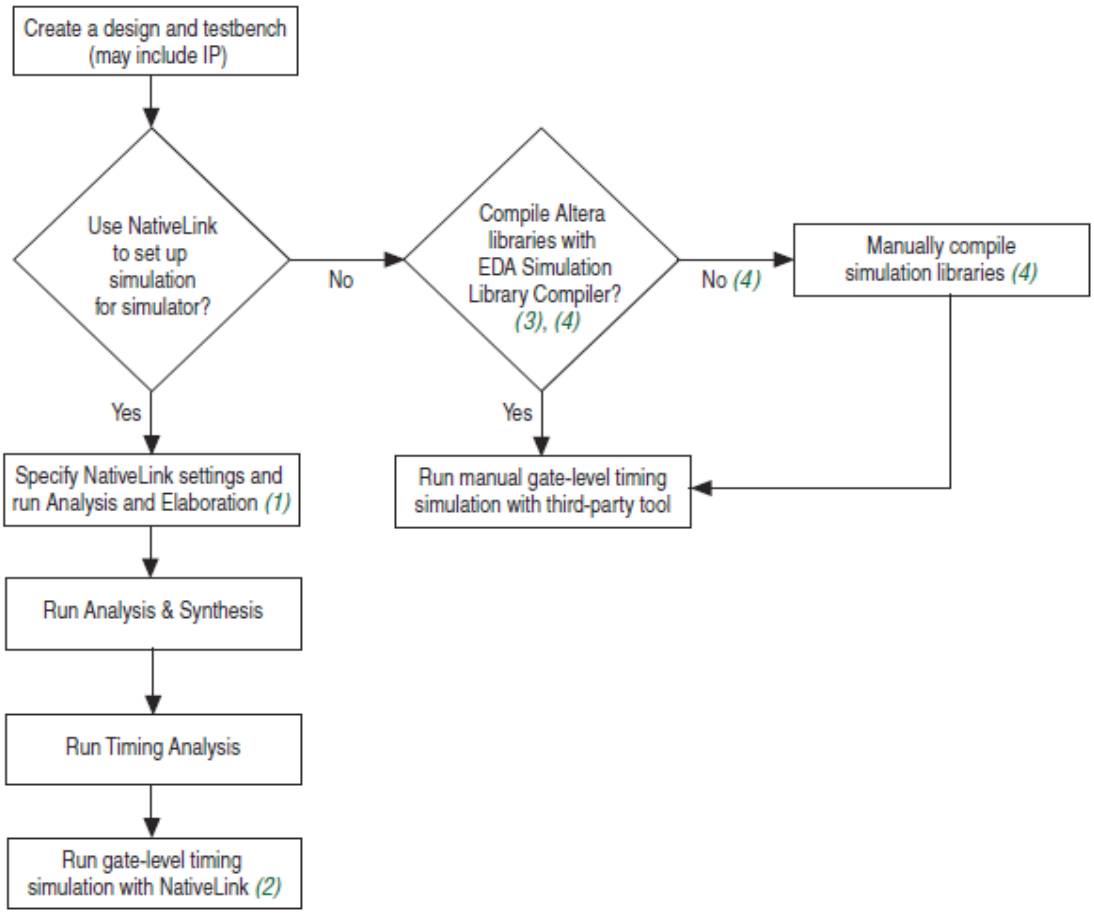
(1) For more information, refer to "Launching the EDA Simulator with the NativeLink Feature" on page 1–9.

(2) For more information, refer to "Running Functional Simulation Using the NativeLink Feature" on page 1–20.

(3) For more information, refer to "EDA Simulation Library Compiler" on page 1–8.

(4) Not applicable for ModelSim AE and ModelSim ASE.

Figure 1–3. Gate-Level Timing Simulation Flow



Notes to Figure 1–3:

(1) For more information, refer to "Launching the EDA Simulator with the NativeLink Feature" on page 1–9.
(2) For more information, refer to "Running Gate-Level Timing Simulation Using the NativeLink Feature" on page 1–21.
(3) For more information, refer to "EDA Simulation Library Compiler" on page 1–8.
(4) Not applicable for ModelSim AE and ModelSim ASE.

Types of Simulations

1. RTL Simulation. It is behavioral simulation that does not target any specific device. It does not model the gate delay and the interconnect delay. The VHDL codes may contain un-synthesizable constructs.
2. Post Synthesis simulation. This is called functional simulation which uses the synthesized gate-level netlist to model the system under test. It uses the gate-level logic expression to do the simulation, but the gate delay and interconnect delay are not included (modeled).
3. Post Synthesis timing simulation. This is called timing simulation which targets a specific device and include both the device specific gate delay and interconnect delay. This simulation takes the most time; as a result, the designer will usually do the functional simulation before attempting to do the timing simulation.

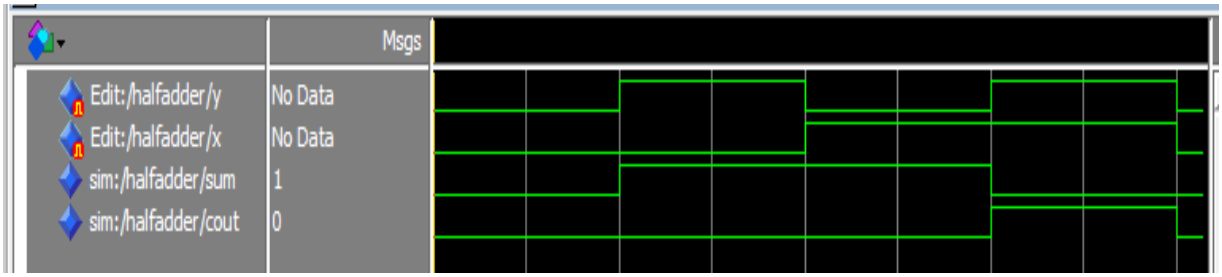
Quartus offers two ways to simulate your design using Modelsim simulator:

1. Perform manual simulation using Modelsim simulator. Tutorial 1 and Tutorial 2 will describe this procedure.

2. Use the NativeLink feature to automatically create scripts for Modelsim simulator and launch it. Tutorial 3 will describe this procedure.

Modelsim supports graphical simulation and testbench

Example: Graphical simulation of half adder.

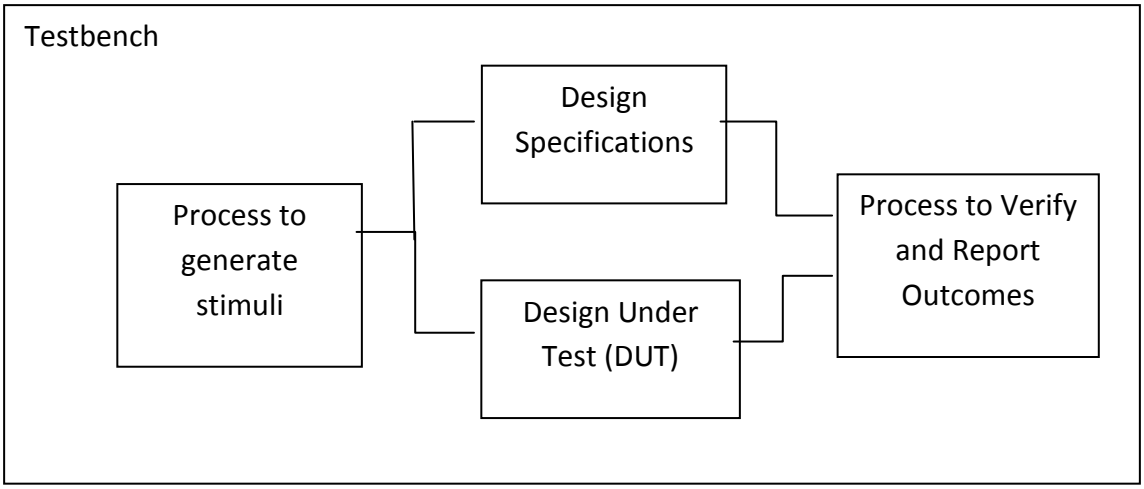


The stimuli which consist of all possible combinations of x and y input values are **manually** set by the user.

Testbench is an HDL program (in our case VHDL) that is used to generate test vectors for the inputs of the component or system under test. Often times, due to time restriction, designing a complete set of test vectors may not be possible; the designer then select the appropriate set of test vectors to exercise the functionality of the component or system under test. If desired, the designer can set up the test bench to monitor the outputs and to issue warnings if the output values are not as expected. Additionally, the designer can manually verify the outputs of the simulated output waveforms.

Testbench is not meant to be synthesized into a circuit; it is primarily used to check if a design-under-test (DUT) is functioning correctly. Hence, Testbench is not restricted to synthesizable VHDL constructs but can use the full set of VHDL constructs: synthesizable and non-synthesizable.

Testbench Structure



Testbench Characteristics

1. Testbench is the TOP design; Design-Under-Test (DUT) is a component in testbench.
2. Testbench normally has no input or output. That is, it has an empty ENTITY.

Testbench VHDL Coding Structure

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;
ENTITY test_my_design IS
END test_my_design;
ARCHITECTURE  behavior of test_my_design IS
    component declaration for my designs;
    other declarations;
BEGIN
    my_design instantiation;          -- component instantiation
    stimulus process;                 -- generate stimuli
    specification process             -- generate values to be compared with my_design
outputs
    check and report process;         --check and report outcomes
END behavior;
```

Test Vectors Generations (see the examples for more details)

1. Constant values: Literals or Vectors.
   X <= '0'; Y <= '1'; Z <= "0001";

2. Expressions
   X <= NOT(X); Z <= Z + 1;

3. Array
   X <= ArrayValue(2);

4. File

Test Vector Coverage

To ensure that the design absolutely satisfies the requirements, the designer must do extensive simulation and the test vectors must cover all possible combinations of input values and internal state of the design. For n-bit inputs and k-bit flip-flops, the test vectors must include 2^{n+k} possible combinations. A system may have a very large number of inputs and internal states. Hence, to test all 2^{n+k} possible combinations may not practically possible. One possible popular case study involves the Intel 64-bit floating point divider. In this case, testing all 2^{n+k} possible combinations on a single computer may take billion years!

Simulation Verification and Reporting

Use ASSERT statement to verify and report outcome.

```
Syntax:   ASSERT condition          -- negative logic
            REPORT "string"         --report when condition is NOT true
            SEVERITY   level        -- optional
                                    -- level options (lowest to highest):
                                    --  NOTE, WARNING, ERROR, FAILURE
```

Example1 :

Test bench for half adder (uses Constant values: Literals or Vectors)

No automated output verification.

Designer needs to verify the outputs on the simulated waveform

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY testhalfadder IS
    -- Empty ENTITY
END testhalfadder;

ARCHITECTURE structure OF testhalfadder IS
SIGNAL x, y : std_logic := '0';
SIGNAL sum, carryout : std_logic;
    COMPONENT HalfAdder
    PORT(  x, y : IN std_logic ;
                sum, carryout : OUT std_logic );
     END COMPONENT;
BEGIN
--Design under test instantiation
   u1: Halfadder PORT MAP (x => x, y=> y, sum => sum, carryout => carryout);
stimulixy : PROCESS
    BEGIN
          WAIT FOR 5 ns;                          -- y = ’0’  x = ’0’
          ASSERT  sum = '0'
          REPORT " sum = 0 + 0 /= 0"
          SEVERITY ERROR;
          ASSERT  carryout = '0'
          REPORT "Carryout /= 0"
          SEVERITY ERROR;

           x <= NOT x;
          WAIT FOR 5 ns;                          -- y = ’0’  x = ’1’
          ASSERT  sum = '1'
          REPORT " sum = 0 + 1 /= 1"
          SEVERITY ERROR;
          ASSERT  carryout = '0'
           REPORT "Carryout /= 0"
          SEVERITY ERROR;

          y <= ‘1’;   x <= ‘0’;
          WAIT FOR 5 ns;                          -- y = ’1’  x = ’0’
                                                  -- error check and report no shown
          x <= NOT x;
          WAIT FOR 5 ns;                          -- y = ’1’  x = ’1’
                                                  -- error check and report no shown
          REPORT  “ Simulation Done!”;
          WAIT;                                    -- halt simulation
    END PROCESS stimulixy;
END structure;
```

Example 2:

Test bench for Two-Bit Adder Version 1

--Uses Constant values: Literals or Vectors

```
--No automated output verification.
--Designer needs to verify the outputs on the simulated waveform
LIBRARY ieee;
USE ieee.std_logic_1164 .all;

ENTITY testadder2bit IS
END testadder2bit ;

ARCHITECTURE testbench OF testadder2bit IS

COMPONENT adder2bit
PORT (x, y   : IN STD_LOGIC_VECTOR (1 DOWNTO 0);
        cin     : IN STD_LOGIC;
        sum   : OUT STD_LOGIC_VECTOR (1 DOWNTO 0);
        cout1          : OUT STD_LOGIC);
END COMPONENT;

SIGNAL DesignIn : STD_LOGIC_VECTOR (4 DOWNTO 0);
SIGNAL DesignOut : STD_LOGIC_VECTOR (2 DOWNTO 0);

BEGIN

--Design under test instantiation
u1: adder2bit PORT MAP( x => DesignIn(2 DOWNTO 1), y => DesignIn(4 DOWNTO 3),
      cin => DesignIn(0), sum => DesignOut(1 DOWNTO 0), cout1 => DesignOut(2) );

Stimuli: PROCESS
BEGIN
          DesignIn <= "00000",                          --cover all possible combinations
                      "00001" AFTER 20 ns,
                      "00010" AFTER 40 ns,
                      "00011" AFTER 60 ns,
                      "00100" AFTER 80 ns,
                      "00101" AFTER 100 ns,
                      "00110" AFTER 120 ns,
                      "00111" AFTER 140 ns,
                      "01000" AFTER 160 ns,
                      "01001" AFTER 180 ns,
                      "01010" AFTER 200 ns,
                      "01011" AFTER 220 ns,
                      "01100" AFTER 240 ns,
                      "01101" AFTER 260 ns,
                      "01110" AFTER 280 ns,
```

```
                    "01111" AFTER 300 ns,
                    "10000" AFTER 320 ns,
                    "10001" AFTER 340 ns,
                    "10010" AFTER 360 ns,
                    "10011" AFTER 380 ns,
                    "10100" AFTER 400 ns,
                    "10101" AFTER 420 ns,
                    "10110" AFTER 440 ns,
                    "10111" AFTER 460 ns,
                    "11000" AFTER 480 ns,
                    "11001" AFTER 500 ns,
                    "11010" AFTER 520 ns,
                    "11011" AFTER 540 ns,
                    "11100" AFTER 560 ns,
                    "11101" AFTER 580 ns,
                    "11110" AFTER 600 ns,
                    "11111" AFTER 620 ns;

    REPORT "Simulation Done!" ;
    WAIT; -- end simulation
END PROCESS;

END testbench;
```

Example 3:

Test Bench for two-bit adder Version 2

```
--Uses Arrays of Records
--Automated output verification
LIBRARY ieee;
USE ieee.std_logic_1164 .all;

ENTITY testadder2bit is
END testadder2bit ;

ARCHITECTURE testbench OF testadder2bit IS

COMPONENT adder2bit
PORT (x, y : IN STD_LOGIC_VECTOR (1 DOWNTO 0);
        cin : IN STD_LOGIC;
        sum : OUT STD_LOGIC_VECTOR (1 DOWNTO 0);
        cout1 : OUT STD_LOGIC);
END COMPONENT;

```

```
CONSTANT NextInputDelay : TIME := 20 ns;
CONSTANT NumOfVectors : INTEGER:= 32;

TYPE vector IS RECORD
	DesignStimulus : STD_LOGIC_VECTOR (4 DOWNTO 0); -- y1 y0 x1 x0 cin
	DesignSpec : STD_LOGIC_VECTOR (2 DOWNTO 0); -- cout sum1 sum0
END RECORD;

TYPE vectorArray is ARRAY (0 TO NumOfVectors - 1) OF vector;

CONSTANT VectorTable : VectorArray :=
(                                -- y1 y0 x1 x0 cin            cout sum1 sum0
	(DesignStimulus => "00000", DesignSpec => "000"), -- Cover all possible
	(DesignStimulus => "00001", DesignSpec => "001"), --    combinations
	(DesignStimulus => "00010", DesignSpec => "001"),
	(DesignStimulus => "00011", DesignSpec => "010"),
	(DesignStimulus => "00100", DesignSpec => "010"),
	(DesignStimulus => "00101", DesignSpec => "011"),
	(DesignStimulus => "00110", DesignSpec => "011"),
	(DesignStimulus => "00111", DesignSpec => "100"),
	(DesignStimulus => "01000", DesignSpec => "001"),
	(DesignStimulus => "01001", DesignSpec => "010"),
	(DesignStimulus => "01010", DesignSpec => "010"),
	(DesignStimulus => "01011", DesignSpec => "011"),
	(DesignStimulus => "01100", DesignSpec => "011"),
	(DesignStimulus => "01101", DesignSpec => "100"),
	(DesignStimulus => "01110", DesignSpec => "100"),
	(DesignStimulus => "01111", DesignSpec => "101"),
	(DesignStimulus => "10000", DesignSpec => "010"),
	(DesignStimulus => "10001", DesignSpec => "011"),
	(DesignStimulus => "10010", DesignSpec => "011"),
	(DesignStimulus => "10011", DesignSpec => "100"),
	(DesignStimulus => "10100", DesignSpec => "100"),
	(DesignStimulus => "10101", DesignSpec => "101"),
	(DesignStimulus => "10110", DesignSpec => "101"),
	(DesignStimulus => "10111", DesignSpec => "110"),
	(DesignStimulus => "11000", DesignSpec => "011"),
	(DesignStimulus => "11001", DesignSpec => "100"),
	(DesignStimulus => "11010", DesignSpec => "100"),
	(DesignStimulus => "11011", DesignSpec => "101"),
	(DesignStimulus => "11100", DesignSpec => "101"),
	(DesignStimulus => "11101", DesignSpec => "110"),
	(DesignStimulus => "11110", DesignSpec => "110"),
	(DesignStimulus => "11111", DesignSpec => "111") );

SIGNAL DesignIn : STD_LOGIC_VECTOR (4 DOWNTO 0);
```

```
SIGNAL DesignOut : STD_LOGIC_VECTOR (2 DOWNTO 0);

BEGIN

--Design under test instatiation
u1: adder2bit PORT MAP( x => DesignIn(2 DOWNTO 1), y => DesignIn(4 DOWNTO 3),
      cin => DesignIn(0), sum => DesignOut(1 DOWNTO 0), cout1 => DesignOut(2) );

Stimuli: PROCESS
BEGIN
   FOR i in 0 TO NumOfVectors -1 LOOP
      DesignIn <= vectorTable (i).DesignStimulus ;
      WAIT FOR NextInputDelay ;
      ASSERT DesignOut = vectorTable (i).DesignSpec
               REPORT "Output incorrect"
               SEVERITY error;
   END LOOP;
   REPORT "Simulation Done!" ;
  WAIT; -- end simulation
END PROCESS;

END testbench;
```

Example 4:

Test Bench for two-bit adder Version 3

```
--Automated Output verification;  Input Stimulus output to message window
--Uses Record/Array;  Report also shows the stimulus vector value
-- Needs my_conversion_package.vhd added to the project:
-- Assignments => Settings => Simulation => Test Benches
-- my_conversion_package.vhd must be the first file in the project.
-- If it is not the first file, use UP or Down Tab to move the file position.

LIBRARY ieee, work;
USE ieee.std_logic_1164 .all;
USE IEEE.STD_LOGIC_ARITH .all;
USE work.my_conversion_package .ALL;

ENTITY testadder2bit IS
END testadder2bit ;

ARCHITECTURE testbench OF testadder2bit IS

COMPONENT adder2bit
PORT (x, y :          IN  STD_LOGIC_VECTOR (1 DOWNTO 0);
```

```
22          cin :           IN  STD_LOGIC;
23          sum :           OUT  STD_LOGIC_VECTOR (1 DOWNTO 0);
24          cout1 :         OUT  STD_LOGIC);
25 END COMPONENT;
26
27 CONSTANT NextInputDelay : TIME := 20 ns;
28 CONSTANT NumOfVectors : INTEGER:= 32;
29
30 TYPE vector IS RECORD
31       DesignStimulus : STD_LOGIC_VECTOR (4 DOWNTO 0); -- y1 y0 x1 x0 cin
32        DesignSpec : STD_LOGIC_VECTOR (2 DOWNTO 0); -- cout sum1 sum0
33 END RECORD;
34
35 TYPE vectorArray is ARRAY (0 TO NumOfVectors - 1) OF vector;
36
37 CONSTANT VectorTable : VectorArray :=
38 (                    -- y1 y0 x1 x0 cin              cout sum1 sum0
39 (DesignStimulus => "00000", DesignSpec => "000"), -- Cover all possible
40 (DesignStimulus => "00001", DesignSpec => "001"), --  combinations
41 (DesignStimulus => "00010", DesignSpec => "001"),
 --Line 42 to line 70 are the same as line 31 to line 59 of example 3. NOT shown here
71
72 SIGNAL DesignIn : STD_LOGIC_VECTOR (4 DOWNTO 0);
73 SIGNAL DesignOut : STD_LOGIC_VECTOR (2 DOWNTO 0);
74
75 BEGIN
76
77 --Design under test instantiation
78 u1: adder2bit PORT MAP( x => DesignIn(2 DOWNTO 1), y => DesignIn(4 DOWNTO 3),
79          cin => DesignIn(0), sum => DesignOut(1 DOWNTO 0), cout1 => DesignOut(2) );
80
81 Stimuli: PROCESS
82 BEGIN
83    FOR i in 0 TO NumOfVectors -1 LOOP
84         DesignIn <= vectorTable (i).DesignStimulus ;
85         WAIT FOR NextInputDelay ;
86         REPORT "input " & tostring(i) & " = " & tostring(vectorTable (i).DesignStimulus );
87         ASSERT DesignOut = vectorTable (i).DesignSpec
88         REPORT "Output incorrect"
89         SEVERITY error;
90   END LOOP;
91   REPORT "Simulation Done!" ;
92   WAIT; -- end simulation
93 END PROCESS;
94
95 END testbench;
```